

Assembler.java

```
1 import java.util.Scanner;
2 import java.util.ArrayList;
3 import java.lang.Math;
4 import java.nio.file.Paths;
5 import java.util.regex.Pattern;
6 //function(Pattern.quote("+"))
7
8 public class Assembler {
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11
12         // ASSEMBLER PROJECT
13         // The file name is Code.asm
14         ArrayList<String> untranslatedCode = new ArrayList<String>();
15         ArrayList<String> translatedCode = new ArrayList<String>();
16         ArrayList<ArrayList<String>> translatedTable = new ArrayList<ArrayList<String>>();
17         try(Scanner fileScanner = new Scanner(Paths.get("Code.asm"))){
18             while(fileScanner.hasNextLine()){
19                 String untranslatedRow = fileScanner.nextLine();
20                 untranslatedCode.add(untranslatedRow);
21             }
22         }
23         catch(Exception e){
24             System.out.println("the file couldnt be accessed");
25             System.out.println("Error is "+ e);
26         }
27         ArrayList<String> refinedCode = new ArrayList<String>();
28         // we will now call the refining function.
29         // to ignore the comments.
30         refinedCode = refiningFunction(untranslatedCode);
31         ArrayList<String> untranslatedCode_new = refinedCode;
32         // printingFunction(untranslatedCode_new);
33         // Now that we have a untranslated code stored in our data structure we have to
34         // make a translation function
35         // We have to handle the Label/Loop variables beforehand;
36         // find some way to transmit the data to the tranlationFunction
37         // Lets send a ArrayList<String>
38         // find some way to transmit the data to the tranlationFunction
39         // Lets send a ArrayList<String>
40         ArrayList<String> labelVariables = new ArrayList<String>();
41         labelVariables = labelGenerationFunction(untranslatedCode_new);
42         translatedTable = translationFunction(untranslatedCode_new, labelVariables);
43         translatedCode = translatedTable.get(0); // CODE --> 0
44         //We need to display our whole variable table AND we also need its value stored in main memory
45 here         ArrayList<String> variableTable = translatedTable.get(1);
46
47         printingFunction(translatedCode);
48         // displayFunction(variableTable);
49
50         /*
51         We have ran into the spacing issue
52         We have to make sure that there are no extra spaces before pressing enter
53         Lets write the code for that character != ' '
54         i.e. if(character == whitespace){break;}
55         i.e. I got a string and i gotta pass that String through a refining function
56         */
57
58         scanner.close();
59     }
60     public static ArrayList<String> labelGenerationFunction(ArrayList<String> untranslatedCode){
61         ArrayList<String> loopVariables = new ArrayList<String>();
62         int i = 0;
63         int counter = 0; //This is the label counter
64         // Used to keep track of the right lines at which label variables point to
65         while(i <= untranslatedCode.size() -1){
66             char fistCharacter = untranslatedCode.get(i).charAt(0);
67             if(fistCharacter == '('){
68                 String labelName = insideBracketFunction(untranslatedCode.get(i));
69                 int location = i;
70                 location = location - counter;
71                 String variableValue = String.valueOf(location);
72                 String variableName = labelName + ";" + variableValue;
73                 loopVariables.add(variableName);
74                 counter++;
75             }
76         }
```

```
76         i++;
77     }
78     return loopVariables;
79 }
80 public static void printingFunction(ArrayList<String> toPrint){
81     for(String row : toPrint){
82         System.out.println(row);
83     }
84 }
85 public static void displayFunction(ArrayList<String> variable){
86     System.out.println("-----");
87     System.out.println("| VARIABLE NAME          |" + " VALUE          |" + " BINARY-VALUE
88 |");
89     System.out.println("-----");
90     String s1 = "VARIABLE NAME          ";
91     int size1 = s1.length();
92     String s2 = "VALUE          ";
93     int size2 = s2.length();
94     String s3 = "BINARY-VALUE          ";
95     int size3 = s3.length();
96     for(int i = 0; i <= variable.size() -1; i++){
97         String[] variableData = variable.get(i).split(";");
98         String variableName = variableData[0];
99         int length1 = size1 - variableName.length();
100        String whiteSpaces1 = whiteSpaceFunction(length1);
101        // variableName = variableName + whiteSpaces1;
102        String variableValue = variableData[1];
103        int value = Integer.valueOf(variableValue);
104        String binaryValue = binaryFunction(value);
105        System.out.print("| " + variableName + whiteSpaces1 + "| ");
106        int length2 = size2 - variableValue.length();
107        String whiteSpaces2 = whiteSpaceFunction(length2);
108        System.out.print(variableValue + whiteSpaces2 + "| ");
109        int length3 = size3 - binaryValue.length();
110        String whiteSpaces3 = whiteSpaceFunction(length3);
111        System.out.print(binaryValue + whiteSpaces3 + "|\n");
112    }
113 }
114 public static String whiteSpaceFunction(int length1){
115     String whiteSpaces = "";
116     for(int i = 1; i <= length1; i++){
117         whiteSpaces = whiteSpaces + " ";
118     }
119     return whiteSpaces;
120 }
121 public static ArrayList<ArrayList<String>> translationFunction(ArrayList<String> untranslatedCode,
ArrayList<String> loopVariables){
122     ArrayList<String> translatedCode = new ArrayList<String>();
123     ArrayList<String> variables = new ArrayList<String>();
124     ArrayList<ArrayList<String>> translatedTable = new ArrayList<ArrayList<String>>();
125     int i = 0;
126     int variableCounter = 16;
127     variables = predefinedAddition();
128     for(int j = 0; j <= loopVariables.size() -1; j++){
129         variables.add(loopVariables.get(j));
130     }
131     while(i <= untranslatedCode.size() -1){
132         char fistCharacter = untranslatedCode.get(i).charAt(0);
133         char secondCharacter = untranslatedCode.get(i).charAt(1);
134         if(fistCharacter == '('){ //Lets deal with loops first 2 if's
135             /*
136             String labelName = insideBracketFunction(untranslatedCode.get(i));
137             int location = i;
138             String variableValue = String.valueOf(location);
139             String variableName = labelName + ";" + variableValue;
140             variables.add(variableName);
141             */
142             i = i + 1;
143             continue;
144         }
145
146         if(fistCharacter == '@' && (secondCharacter == '0' || secondCharacter == '1' ||
secondCharacter == '2' ||
147 secondCharacter == '3' || secondCharacter == '4' || secondCharacter == '5' || secondCharacter
== '6' ||
```

```

148         secondCharacter == '7' || secondCharacter == '8' || secondCharacter == '9')){
149             //checking if it is a number or a variable at second place.
150             String translatedRow = binaryFunction(untranslatedCode.get(i));
151             translatedCode.add(translatedRow);
152         }
153         else if(fistCharacter == '@' && (secondCharacter != '0' || secondCharacter != '1' ||
154             secondCharacter != '2' || secondCharacter != '3' || secondCharacter != '4' ||
155             secondCharacter != '5' || secondCharacter != '6' || secondCharacter != '7' ||
156             secondCharacter != '8' || secondCharacter != '9')){
157             String[] temporary = untranslatedCode.get(i).split("@");
158             String translatedRow;
159             String variableName = temporary[1];
160
161             if(variableCheckFunction(variables, variableName) == false){
162                 variableName = variableName + ";" + String.valueOf(variableCounter); //CSV
163                 // CSV --> Comma Separated Values
164                 variables.add(variableName);
165                 translatedRow = variableTranslationFunction(variables.get(variables.size()-1));
166                 variableCounter++; // we start from sixteen
167             }
168
169             else{
170                 translatedRow = alreadyExistingFunction(variables, variableName);
171             }
172             translatedCode.add(translatedRow);
173         }
174         else{
175             Assembler assembler = new Assembler();
176             Assembler.ControlBits controls = assembler.new ControlBits(untranslatedCode.get(i));
177             String translatedRow = controls.Command;
178             translatedCode.add(translatedRow);
179         }
180         i++;
181     }
182     translatedTable.add(translatedCode); // 0 --> CODE
183     translatedTable.add(variables);      // 1 --> TABLE
184     return translatedTable;
185 }
186 public static String insideBracketFunction(String label){
187     int i = 0;
188     char characterValue = label.charAt(i);
189     String name = "";
190     while(true){
191         i++;
192         characterValue = label.charAt(i);
193         if(characterValue == ' '){
194             break;
195         }
196         name = name + label.charAt(i);
197     }
198     return name;
199 }
200 public static boolean variableCheckFunction(ArrayList<String> variables, String variableName){
201     boolean checker = false;
202     for(int i = 0; i <= variables.size() - 1; i++){
203         String temporary = variables.get(i);
204         String[] temporary1 = temporary.split(";");
205         String temporary2 = temporary1[0];
206         if(temporary2.equals(variableName)){
207             checker = true;
208         }
209     }
210     return checker;
211 }
212 public static String alreadyExistingFunction(ArrayList<String> variables, String variableName){
213     String translatedRow;
214     int value = 0;
215     for(int i = 0; i <= variables.size() - 1; i++){
216         String[] temporary = variables.get(i).split(";");
217         String temporary1 = temporary[0];
218         String value1 = temporary[1];
219         if(temporary1.equals(variableName)){
220             value = Integer.valueOf(value1);
221             break;
222         }
223     }
224     translatedRow = binaryFunction(value);
225     return translatedRow;

```

```

226     }
227
228     public static String variableTranslationFunction(String untranslatedRow){
229         String translatedRow;
230         String[] temporary = untranslatedRow.split(";");
231         translatedRow = temporary[1];
232         String translatedBinaryRow = binaryFunction(Integer.valueOf(translatedRow));
233         return translatedBinaryRow;
234     }
235
236     public static ArrayList<String> refiningFunction(ArrayList<String> untranslatedCode){
237         ArrayList<String> refinedCode = new ArrayList<String>();
238         int i = 0;
239         while(i <= untranslatedCode.size() -1){
240             if(untranslatedCode.get(i).equals("") != true){ // now we can ignore blankLines
241                 if(untranslatedCode.get(i).charAt(0) != '/'){
242                     // we are using this function to ignore the comments in the code
243                     String row = RefineryFunction(untranslatedCode.get(i));
244                     /*
245                         if(row.contains("A") || row.contains("D") || row.contains("M") ||
246                         row.contains("@") || row.contains("(") || row.contains("=") ||
247                         row.contains("!") || row.contains("&") || row.contains("|") ||
248                         row.contains("l") || row.contains("0") || row.contains("-l") ||
249                         row.contains("J") || row.contains("N") || row.contains("-") ||
250                         row.contains(Pattern.quote("+"))){ */
251                         refinedCode.add(row);
252                     // REVEALATION - Empty Lines are handled Automatically}
253                 }
254             }
255             i++;
256         }
257         return refinedCode;
258     }
259
260     public static String RefineryFunction(String original){
261         String updated = ""; // NULL is unpredictable
262         int i = 0;
263         while(i <= original.length() -1){
264             // If the Code was using ; it would have been Easy.
265             // This also means no white-space use in the code allowed
266             // f.y.i. empty lines are also not allowed .contains('|'|'|'|'|')
267             // Revelation: Empty lines are handled automatically , idk man really idk it works!
268             // Alright Let me also allow empty lines
269             if(original.charAt(i) == ' '){
270                 break;
271             }
272             updated = updated + original.charAt(i);
273             i++;
274         }
275         return updated;
276     }
277
278     public static String binaryFunction(String untranslatedRow){
279
280         String translatedRow = "";
281         String[] temporarySplitter = untranslatedRow.split("@");
282         String valueStr = temporarySplitter[1];
283         int value = Integer.valueOf(valueStr);
284         int quotient = (value / 2);
285         int remainder = (value % 2);
286         int counter = 1;
287         value = quotient;
288         //int placeCounter = 0; //first we are operating at the base of 10
289         //int placeValue = (int)Math.pow(10, placeCounter); //typeCasting double --> integer
290         //int binaryValue = remainder*placeValue;
291         translatedRow = String.valueOf(remainder) + translatedRow;
292         while(quotient != 0){
293             quotient = value/2;
294             remainder = value%2;
295             value = quotient;
296             translatedRow = String.valueOf(remainder) + translatedRow;
297             counter++;
298         }
299         String zeros = "";
300         for(int i = 1; i <= 16-(counter+1); i++){
301             zeros = zeros + "0";
302         }
303         translatedRow = "0" + zeros + translatedRow;
304         return translatedRow;
305     }
306 }
307

```

```
304 public static String binaryFunction(int value){
305     String binaryTranslation = "";
306     int quotient = (value / 2);
307     int remainder = (value % 2);
308     int counter = 1;
309     value = quotient;
310     binaryTranslation = String.valueOf(remainder) + binaryTranslation;
311     while(quotient != 0){
312         quotient = value/2;
313         remainder = value%2;
314         value = quotient;
315         binaryTranslation = String.valueOf(remainder) + binaryTranslation;
316         counter++;
317     }
318     String zeros = "";
319     for(int i = 1; i <= 16-(counter+1); i++){
320         zeros = zeros + "0";
321     }
322     binaryTranslation = "0" + zeros + binaryTranslation;
323     return binaryTranslation;
324 }
325 public static ArrayList<String> predefinedAddition(){
326     ArrayList<String> keyWords = new ArrayList<String>();
327     String[] keywords = {"R0;0", "R1;1", "R2;2", "R3;3", "R4;4", "R5;5", "R6;6", "R7;7",
328         "R8;8", "R9;9", "R10;10", "R11;11", "R12;12", "R13;13", "R14;14",
329         "R15;15", "SCREEN;16384", "KBD;24576", "SP;0", "LCL;1", "ARG;2",
330         "THIS;3", "THAT;4" };
331
332     for(int i = 0; i <= keywords.length -1; i++){
333         keyWords.add(keywords[i]);
334     }
335     return keyWords;
336 }
337
338 public class ControlBits{
339     private String[] opCode = {"0", "1"}; //a
340     private String[] computation = {"0;101010", "1;11111", "-1;111010", "D;001100", "A;110000",
341         "!D;001101", "!A;110001", "-D;001111", "-A;110011",
342         "D+1;011111", "A+1;110111", "D-1;001110", "A-1;110010",
343         "D+A;000010", "D-A;010011", "A-D;000111", "D&A;000000",
344         "D|A;010101" }; //ccccc
345     private String[] destination = {"NULL;000", "M;001", "D;010", "DM;011", "A;100", "AM;101",
346         "AD;110", "ADM;111"}; //ddd
347     private String[] jumper = {"NULL;000", "JGT;001", "JEQ;010", "JGE;011", "JLT;100",
348         "JNE;101", "JLE;110", "JMP;111"}; //jjj
349     public String Command;
350     public ControlBits(String Command){
351         this.Command = commandFunction(Command);
352     }
353     public String commandFunction(String Command){
354         String finalCommand = "";
355         boolean checkSemicolon = Command.contains(";");
356         boolean checkM;
357         // boolean checkM = Command.contains("M"); misleading{just here so I can learn from my
errors}
358         boolean checkEqual = Command.contains("=");
359         String Destination;
360         String Jumper;
361         String Computation;
362         String opCode;
363         if(checkSemicolon && checkEqual){
364             String[] initial = Command.split(";");
365             String jumper = initial[1];
366             Jumper = jumperFunction(jumper);
367             String other = initial[0];
368             String[] information = other.split("=");
369             String destination = information[0];
370             Destination = destinationFunction(destination);
371             String computation = information[1];
372             checkM = computation.contains("M");
373             Computation = computationFunction(computation, checkM);
374             opCode = "0"; //default
375             if(checkM){
376                 opCode = "1";
377             }
378             finalCommand = "111" + opCode + Computation + Destination + Jumper; // 111acccccddjjj
379         }
380         else if(checkSemicolon == true && checkEqual == false){
381             String[] information = Command.split(";");
```

```

382         String computation = information[0];
383         String jumper = information[1];
384         Destination = "000"; //NULL
385         checkM = computation.contains("M");
386         Computation = computationFunction(computation, checkM);
387         Jumper = jumperFunction(jumper);
388         opCode = "0";
389         if(checkM){
390             opCode = "1";
391         }
392         finalCommand = "111" + opCode + Computation + Destination + Jumper; // 111acccccddjjj
393     }
394     else if(checkSemicolon == false && checkEqual == true){
395         String[] information = Command.split("=");
396         String destination = information[0];
397         String computation = information[1];
398         checkM = computation.contains("M");
399         Jumper = "000"; //i.e. NULL
400         Destination = destinationFunction(destination);
401         Computation = computationFunction(computation, checkM);
402         opCode = "0";
403         if(checkM){
404             opCode = "1";
405         }
406         finalCommand = "111" + opCode + Computation + Destination + Jumper; // 111acccccddjjj
407     }
408     return finalCommand;
409 }
410 public String jumperFunction(String jumper){
411     String binaryTranslation="";
412     for(int i = 0; i <= this.jumper.length -1; i++){
413         String[] information = this.jumper[i].split(";");
414         String instruction = information[0];
415         String value = information[1];
416         if(jumper.equals(instruction)){
417             binaryTranslation = value;
418             break;
419         }
420     }
421     return binaryTranslation;
422 }
423 public String destinationFunction(String destination){
424     String binaryTranslation="";
425     for(int i = 0; i <= this.destination.length -1; i++){
426         String[] information = this.destination[i].split(";");
427         String instruction = information[0];
428         String value = information[1];
429         if(destination.equals(instruction)){
430             binaryTranslation = value;
431             break;
432         }
433     }
434     return binaryTranslation;
435 }
436 public String computationFunction(String computation, boolean checkM){
437     String computation_new;
438     String binaryTranslation = "";
439     if(checkM){
440         computation_new = replaceFunction(computation);
441     }
442     else{
443         computation_new = computation;
444     }
445     String inverseString = "";
446     if(computation_new.contains("+") || computation_new.contains("-") ||
447        computation_new.contains("&") || computation_new.contains("|")){
448         inverseString = inverseFunction(computation_new);
449     }
450     for(int i = 0; i <= this.computation.length -1; i++){
451         String[] information = this.computation[i].split(";");
452         String instruction = information[0];
453         String value = information[1];
454         if(instruction.contains("-")){
455             if(instruction.equals(computation_new)){
456                 binaryTranslation = value;
457                 break;
458             }
459         }

```

```
460         else{
461             if(instruction.equals(computation_new) || instruction.equals(inverseString)){
462                 binaryTranslation = value;
463                 break;
464             }
465         }
466     }
467     return binaryTranslation;
468 }
469 public String inverseFunction(String computation_new){
470     String inverseString; // A+D --> D+A; +,|,&,A-D --> D-A
471     String middle = "";
472     if(computation_new.contains("+")){
473         middle = "+";
474     }
475     else if(computation_new.contains("-")){
476         middle = "-";
477     }
478     else if(computation_new.contains("&")){
479         middle = "&";
480     }
481     else if(computation_new.contains("|")){
482         middle = "|";
483     }
484     // middle = "\\\" +middle; Regex
485     String[] twoSides = computation_new.split(Pattern.quote(middle));
486     String side1 = twoSides[0];
487     String side2 = twoSides[1];
488     inverseString = side2 + middle + side1;
489     return inverseString;
490 }
491 public String replaceFunction(String computation){
492     String replacement = "";
493     int mIndicator = 77;
494     String[] storage = new String[2];
495     storage[0] = ""; // Be very very careful with NULL values
496     storage[1] = "";
497     int m = 0; // storage parts
498     for(int i = 0; i <= computation.length() -1; i++){
499         if(computation.charAt(i) == 'M'){
500             // storage[m] = storage[m] + String.valueOf(computation.charAt(i));
501             mIndicator = 77;
502             m++;
503             continue;
504         }
505         else{
506             storage[m] = storage[m] + String.valueOf(computation.charAt(i));
507         }
508     }
509     if(mIndicator == 77){
510         replacement = storage[0] + "A" + storage[1];
511     }
512     else{
513         replacement = computation;
514     }
515     return replacement;
516 }
517 }
518 }
519
520
```